

This Java 8 challenge tests your knowledge of [Lambda expressions](#)!

Write the following methods that return a lambda expression performing a specified action:

1. PerformOperation isOdd(): The lambda expression must return *true* if a number is odd or *false* if it is even.
2. PerformOperation isPrime(): The lambda expression must return *true* if a number is prime or *false* if it is composite.
3. PerformOperation isPalindrome(): The lambda expression must return *true* if a number is a palindrome or *false* if it is not.

Input Format

Input is handled for you by the locked stub code in your editor.

Output Format

The locked stub code in your editor will print *T* lines of output.

Sample Input

The first line contains an integer, *T* (the number of test cases).

The *T* subsequent lines each describe a test case in the form of 2 space-separated integers:

The first integer specifies the condition to check for (1 for Odd/Even, 2 for Prime, or 3 for Palindrome). The second integer denotes the number to be checked.

Upload Code as File

Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1 5
2 1 4
3 2 5
4 3 898
5 1 3
6 2 12
```

Download

Your Output (stdout)

```
1 EVEN
2 PRIME
3 PALINDROME
4 ODD
```

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The Player class is provided for you in your editor. It has **2** fields: a *name* String and a *score* integer.

Given an array of *n* Player objects, write a comparator that sorts them in order of decreasing score; if **2** or more players have the same score, sort those players alphabetically by name. To do this, you must create a Checker class that implements the Comparator interface, then write an int compare(Player a, Player b) method implementing the `Comparator.compare(T o1, T o2)` method.

Input Format

Input from stdin is handled by the locked stub code in the Solution class.

The first line contains an integer, *n*, denoting the number of players.

Each of the *n* subsequent lines contains a player's *name* and *score*, respectively.

Constraints

- $0 \leq \text{score} \leq 1000$
- 2 players can have the same name.
- Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to stdout. The locked stub code in Solution will create a Checker object, use it to sort the Player array, and print each sorted element.

[Upload Code as File](#)☐ Test against custom input[Run Code](#)[Submit Code](#)

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
```

[Download](#)

Your Output (stdout)

```
aleksa 150
amy 100
david 100
aakansha 75
```



You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

Hint: You can use comparators to sort a list of objects. See the [oracle docs](#) to learn about comparators.

Input Format

The first line of input contains an integer N , representing the total number of students. The next N lines contains a list of student information in the following structure:

```
ID Name CGPA
```

Constraints

$$2 \leq N \leq 1000$$

$$0 \leq ID \leq 100000$$

$$5 \leq |Name| \leq 30$$

$$0 \leq CGPA \leq 4.00$$

The name contains only lowercase English letters. The ID contains only integer numbers without leading zeros. The CGPA will contain, at most, 2 digits after the decimal point.

Output Format

After rearranging the students according to the above rules, print the first name of each

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#)

[Next Challenge](#)

✓ Test case 0

✓ Test case 1 [lock](#)

✓ Test case 2 [lock](#)

✓ Test case 3 [lock](#)

✓ Test case 4 [lock](#)

✓ Test case 5 [lock](#)

✓ Test case 6 [lock](#)

```
5
33 Rumpa 3.68
85 Ashis 3.85
56 Samiha 3.75
19 Samara 3.75
22 Fahim 3.76
```

Expected Output

```
Ashis
Fahim
Samara
Samiha
Rumpa
```

[Download](#)
[Show desktop](#)


You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

Hint: You can use comparators to sort a list of objects. See the [oracle docs](#) to learn about comparators.

Input Format

The first line of input contains an integer N , representing the total number of students. The next N lines contains a list of student information in the following structure:

```
ID Name CGPA
```

Constraints

$$2 \leq N \leq 1000$$

$$0 \leq ID \leq 100000$$

$$5 \leq |Name| \leq 30$$

$$0 \leq CGPA \leq 4.00$$

The name contains only lowercase English letters. The ID contains only integer numbers without leading zeros. The CGPA will contain, at most, 2 digits after the decimal point.

Output Format

After rearranging the students according to the above rules, print the first name of each

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#)

[Next Challenge](#)

✓ Test case 0

✓ Test case 1 [lock](#)

✓ Test case 2 [lock](#)

✓ Test case 3 [lock](#)

✓ Test case 4 [lock](#)

✓ Test case 5 [lock](#)

✓ Test case 6 [lock](#)

```
5
33 Rumpa 3.68
85 Ashis 3.85
56 Samiha 3.75
19 Samara 3.75
22 Fahim 3.76
```

Expected Output

```
Ashis
Fahim
Samara
Samiha
Rumpa
```

[Download](#)
[Show desktop](#)


Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The Player class is provided in the editor below. It has two fields:

1. *name*: a string.
2. *score*: an integer.

Given an array of *n* Player objects, write a comparator that sorts them in order of decreasing score. If 2 or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a Checker class that implements the Comparator interface, then write an int compare(Player a, Player b) method implementing the `Comparator.compare(T o1, T o2)` method. In short, when sorting in ascending order, a comparator function returns -1 if $a < b$, 0 if $a = b$, and 1 if $a > b$.

Declare a Checker class that implements the comparator method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of Player objects, uses your method to sort the data, and prints it out properly.

Example

$n = 3$ data = `[[Smith, 20], [Jones, 15], [Jones, 20]]`

Sort the list as `datasorted = [[Jones, 20], [Smith, 20], [Jones, 15]]`. Sort first descending by score, then ascending by name.

Input Format

The first line contains an integer, *n*, the number of players.

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#)

[Next Challenge](#)

✓ Test case 0

✓ Test case 1

✓ Test case 2

✓ Test case 3

✓ Test case 4

✓ Test case 5

✓ Test case 6

```
1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakansha 75
6 aleksa 150
```

Expected Output

```
1 aleksa 150
2 amy 100
3 david 100
4 aakansha 75
5 heraldo 50
```

[Download](#)

English (United States)
English (India)

To switch input methods, press Windows key + space.

1480. Running Sum of 1d Array

Easy Topics Companies Hint

Given an array `nums`. We define a running sum of `nums` as `runningSum[i] = sum(nums[0]...nums[i])`. Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`
Output: `[1,3,6,10]`
Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`
Output: `[1,2,3,4,5]`
Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

8.9K 218

Code

Java Auto

```
1 class Solution {
2     public int[] runningSum(int[] nums) {
3
4         for (int i = 1; i < nums.length; i++) {
5
6             nums[i] = nums[i - 1] + nums[i];
7         }
8
9         return nums;
10    }
11 }
```

Saved

Ln 3, Col 9

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[1,2,3,4]

Output

[1,3,6,10]

Expected

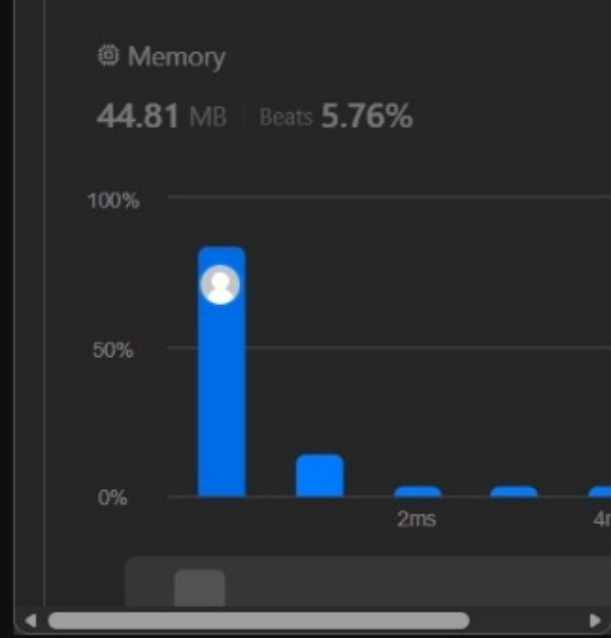
Description Accepted x Editorial

All Submissions

Accepted 34 / 34 testcases passed
srikiran57 submitted at Jul 28, 2026 11:34

₹583.25/mo Save over 56%
Unlock the Full LeetCode...
Company problems, Ask Leet, and...

Runtime
0 ms | Beats 100.00%



</> Code

```
Java Auto  
2 public int maximumWealth(int[][] accounts) {  
3     int maxWealth = 0;  
4  
5     for (int[] customer : accounts) {  
6         int currentWealth = 0;  
7  
8         for (int bank : customer) {  
9             currentWealth += bank;  
10        }  
11    }  
12 }
```

Saved Ln 4, Col 5

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

accounts =
[[1,2,3], [3,2,1]]

Output

6

Expected

https://leetcode.com/problems/richest-customer-wealth/description/

Problem List

Accepted

1672. Richest Customer Wealth

Easy

Topics

Companies

Hint

You are given an $m \times n$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return the **wealth** of the richest customer has.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**.

Example 1:

Input: `accounts = [[1,2,3],[3,2,1]]`
Output: 6
Explanation:
1st customer has wealth = 1 + 2 + 3 = 6
2nd customer has wealth = 3 + 2 + 1 = 6
Both customers are considered the richest with a wealth of 6 each, so return 6.

Example 2:

4.9K

108

Star

Share

Help

Code

Java

Auto

2

3

4

5

6

7

8

9

10

11

12

public int maximumWealth(int[][] accounts) {
 int maxWealth = 0;

 for (int[] customer : accounts) {
 int currentWealth = 0;

 for (int bank : customer) {
 currentWealth += bank;
 }
 }
}

Saved

Ln 4, Col 5

Submit

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

accounts =
[[1,2,3],[3,2,1]]

Output

6

Expected

3 34°C
Mostly cloudy

Search

ENG
IN

11:35 AM
7/28/2026

https://leetcode.com/problems/richest-customer-wealth/description/

Problem List

Accepted

1672. Richest Customer Wealth

Easy

Topics

Companies

Hint

You are given an $m \times n$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return the **wealth** of the richest customer has.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**.

Example 1:

Input: `accounts = [[1,2,3],[3,2,1]]`
Output: 6
Explanation:
1st customer has wealth = 1 + 2 + 3 = 6
2nd customer has wealth = 3 + 2 + 1 = 6
Both customers are considered the richest with a wealth of 6 each, so return 6.

Example 2:

4.9K

108

Star

Share

Help

Code

Java

Auto

2

3

4

5

6

7

8

9

10

11

12

public int maximumWealth(int[][] accounts) {
 int maxWealth = 0;

 for (int[] customer : accounts) {
 int currentWealth = 0;

 for (int bank : customer) {
 currentWealth += bank;
 }
 }
}

Saved

Ln 4, Col 5

Submit

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

accounts =
[[1,2,3],[3,2,1]]

Output

6

Expected

3 34°C
Mostly cloudy

Search

ENG IN

11:35 AM
7/28/2026

977. Squares of a Sorted Array

Easy Topics Companies

Given an integer array `nums` sorted in **non-decreasing** order, return an array of **the squares of each number** sorted in non-decreasing order.

Example 1:

Input: `nums = [-4,-1,0,3,10]`

Output: `[0,1,9,16,100]`

Explanation: After squaring, the array becomes `[16,1,0,9,100]`. After sorting, it becomes `[0,1,9,16,100]`.

Example 2:

Input: `nums = [-7,-3,2,3,11]`

Output: `[4,9,9,49,121]`

Constraints:

👍 10.4K 🔄 194 | ☆ 📄 ?

</> Code

Java 🔒 Auto

```
1 class Solution {
2     public int[] sortedSquares(int[] nums) {
3         int n = nums.length;
4         int[] result = new int[n];
5
6
7         int left = 0;
8         int right = n - 1;
9
10
11        for (int i = n - 1; i >= 0; i--) {
```

Saved

Ln 6, Col 9

☑️ Testcase | > Test Result

Accepted Runtime: 0 ms

☑️ Case 1 ☑️ Case 2

Input

nums =
[-4,-1,0,3,10]

Output

[0,1,9,16,100]

Expected

724. Find Pivot Index

Easy Topics Companies Hint

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left of that index. The same applies to the right edge of the array.

Return the **leftmost pivot index**. If no such index exists, return `-1`.

Example 1:

Input: `nums = [1,7,3,6,5,6]`

Output: 3

Explanation:

The pivot index is 3.

Left sum = `nums[0] + nums[1] + nums[2]`
= `1 + 7 + 3` = 11

Right sum = `nums[4] + nums[5]` = 5

9.5K 273

Code

Java Auto

```
1 class Solution {
2     public int pivotIndex(int[] nums) {
3         int totalSum = 0;
4
5
6         for (int num : nums) {
7             totalSum += num;
8         }
9
10        int leftSum = 0;
11    }
```

Saved

Ln 5, Col 9

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[1,7,3,6,5,6]

Output

3

Expected